

Guide: Embedding Binaries in Bash Scripts

This guide provides comprehensive examples and recipes for embedding binary files—such as music, videos, images, or even other executables—directly into a Bash script. This technique is useful for creating self-contained, portable scripts that don't rely on external assets.

1. The Core Technique: Base64 Encoding

Binary files contain non-text characters that can corrupt a script file if pasted directly. To safely embed a binary inside a text-based script, we must first encode it into a text-only format. **Base64** is the standard for this. It represents binary data using only 64 common ASCII characters, making it safe to include anywhere.

The downside is that Base64 encoding increases the file size by approximately 33%.

Step 1: Convert Your File to a Base64 String

Use the `base64` command-line tool. The `-w 0` flag is crucial as it prevents line wrapping, giving you a single, unbroken string to embed.

```
# Syntax: base64 -w 0 <input_file> > <output_file>
```

```
# Example with an audio file
```

```
base64 -w 0 my_song.mp3 > my_song.b64
```

```
# Example with a standalone executable
```

```
base64 -w 0 my_app > my_app.b64
```

2. Embedding Methods

Once you have your Base64 string, you can embed it in your script.

Method A: Simple Variable Assignment

This is the most straightforward method, perfect for smaller files. You assign the entire Base64 string to a variable.

extractor_script.sh

```
#!/bin/bash
#
# A simple script to extract an embedded MP3 file.
#

# The output filename for the extracted file.
OUT_FILE="song_from_script.mp3"

# --- EMBEDDED DATA ---
# The Base64 string is assigned to a variable.
# Note: This string is heavily truncated for this example. In a
real script,
# this would be thousands or millions of characters long.
B64_MUSIC="SUQzBAAAAAAAI1RTU0UAAAAPAAADTGF2ZjU4LjI5LjEwMAAAAAAAAAAAAAAA//
uQxA..."

# --- EXTRACTION LOGIC ---
echo "Extracting music file to '$OUT_FILE'..."

# Use the 'base64 -d' command to decode the string and redirect to
a file.
echo "$B64_MUSIC" | base64 -d > "$OUT_FILE"

# Check if the extraction was successful
if [ $? -eq 0 ]; then
    echo "Extraction complete!"
    echo "You can now play the file: mpv '$OUT_FILE'"
else
    echo "Error: Extraction failed." >&2
    exit 1
fi

# Optionally, try to play the file automatically.
# We check for common media players.
if command -v mpv &> /dev/null; then
    mpv "$OUT_FILE"
elif command -v vlc &> /dev/null; then
    vlc "$OUT_FILE"
elif command -v paplay &> /dev/null; then
    paplay "$OUT_FILE"
fi
```

Method B: Using a Heredoc (Recommended for Larger Files)

When the Base64 string is very long (e.g., >100KB), storing it in a variable can make the script slow to load in editors and can sometimes hit shell limits. A **Heredoc** is a much better way to handle large, multi-line blocks of text.

extractor_heredoc.sh

```
#!/bin/bash
#
# This script uses a Heredoc to embed and extract a binary.
# This method is more robust for larger files.
#

OUT_FILE="binary_from_heredoc.zip"

echo "Extracting binary from Heredoc to '$OUT_FILE'..."

# The 'cat << EOF' syntax starts the Heredoc.
# All lines until 'EOF' are fed as standard input to the 'cat'
command.
# We pipe this directly to 'base64 -d'.
base64 -d > "$OUT_FILE" << EOF
UESDBBQACAAIAAAAAAAAAAAAAAAAAAAAAAAAAAKABwAZGF0YS50eHRVVAAkAA+B6pWXgeLZldXgLA
AAAAAQAAAAATWuIGlzIGRpc3Rpbmd1aXshZWQsIG5vdCBvbmx5IGJ5IGhpcyByZWZzb24sIG
dCBieSB0aGlzIHNpbmd1bGFyIHBoY3Npb24gZnJvbSBvdGhlciBhbmltYWxzLCB3aGljaCBpcy
IGxlc3Qgb2YgdGhlIGlpbmQsIHRoYXQgYnkgYSBwZXJzZXZlcmFuY2Ugb2YgZGVsaWdodCBpb
aGUgY29udGluZGVkIGFuZCBpbmRlZmF0aWdhYm91IGdlbmV5YXRpb24gb2Yga25vd2x1ZGdlL
eGNlZWRRZiIHRoZSBzaG9ydCB2ZWhlbWVudG9ydG9ydG9ydG9ydG9ydG9ydG9ydG9ydG9ydG
eGNlZWRRZiIHRoZSBzaG9ydCB2ZWhlbWVudG9ydG9ydG9ydG9ydG9ydG9ydG9ydG9ydG9ydG
AAGACAAAAAAAAAAAAAAAAAAAAAAAAAACgAYAAAAAAAAAAAAAAGAAAAAAAAABkYXRhLnR4dFVUBQAD4
ZXV4CwABBAAAAAAEAAAAAFBLBQYAAAAAAQABAEAAAAA+AAAAAA=
EOF
# The 'EOF' marker must be on a line by itself with no leading
spaces.

if [ $? -eq 0 ]; then
    echo "Extraction complete!"
    echo "File '$OUT_FILE' created."
    unzip -l "$OUT_FILE" # List contents of the created zip
else
    echo "Error: Extraction failed." >&2
    exit 1
fi
```

3. Handling Larger Files: Compression

Base64 makes files larger. To counteract this, you can **compress** the file before encoding it. `gzip` is perfect for this. The extraction process then becomes a two-step pipe: `base64 -> gzip`.

Recipe: Compress, Encode, Embed, Decode, Decompress

1. **Compress the original file:** `bash` *# The -c flag directs output to stdout, which we redirect.* `gzip -c my_app > my_app.gz`

2. **Base64 encode the compressed file:** `bash` `base64 -w 0 my_app.gz > my_app.gz.b64`

3. **Embed the `.gz.b64` content and update the script logic:**

extractor_compressed.sh

```
#!/bin/bash
# Extracts a gzipped and base64-encoded binary.

OUT_FILE="my_app_extracted"
B64_GZIPPED_DATA="..." # Paste the content of my_app.gz.b64
here

echo "Extracting and decompressing to '$OUT_FILE'..."

# Pipeline:
# 1. Echo the base64 data
# 2. Decode it with 'base64 -d'
# 3. Decompress it with 'gzip -d'
# 4. Redirect the final binary output to our file
echo "$B64_GZIPPED_DATA" | base64 -d | gzip -d > "$OUT_FILE"

# Make it executable (if it's a program)
chmod +x "$OUT_FILE"

echo "Done. You can now run the program: ./\$OUT_FILE"
```

4. Obfuscation with Encryption (Optional)

If you want to prevent someone from easily seeing the embedded content, you can add a layer of encryption using `openssl`.

Disclaimer: This is **not** for high security. The password is still stored in the script, making it trivial to decrypt for anyone who can read the script. This is best for simple obfuscation.

Recipe: Encrypt, Encode, Embed, Decode, Decrypt

1. **Encrypt the file with openssl:** bash # This command encrypts the file using AES-256 and prompts for a password. # To script it, use the -pass flag. openssl enc -aes-256-cbc -salt -in my_secret.data -out my_secret.data.enc -pass pass:'my-secret-password'

2. **Base64 encode the encrypted file:** bash base64 -w 0
my_secret.data.enc > my_secret.data.enc.b64

3. **Update the script to handle decryption:**

extractor_encrypted.sh

```
#!/bin/bash
# Extracts and decrypts an embedded file.

OUT_FILE="my_secret_revealed.data"
PASSWORD="my-secret-password"
B64_ENCRYPTED_DATA="..." # Paste the content of
my_secret.data.enc.b64 here

echo "Extracting and decrypting '$OUT_FILE'..."

# Pipeline:
# 1. Echo the base64 data
# 2. Decode it with 'base64 -d'
# 3. Decrypt it with 'openssl', providing the password
echo "$B64_ENCRYPTED_DATA" | base64 -d | openssl enc -d -
aes-256-cbc -pass pass:"$PASSWORD" > "$OUT_FILE"

if [ $? -eq 0 ]; then
    echo "Success! Secret data extracted."
    cat "$OUT_FILE"
else
    echo "Error: Decryption failed. Wrong password or corrupted
data." >&2
fi
```

5. Advanced: The Self-Extracting Archive

A classic technique is to append the binary data to the script file itself, creating a single-file distributable. The script is written to be able to read itself, find where the payload begins, and extract it.

Recipe: Create a Self-Extracting Script

1. Create the main script logic (make_archive.sh):

```
#!/bin/bash
# This is the script portion of our archive.

# Use grep to find the line number where the payload starts.
# -n: print line number
# -a: process a binary file as if it were text
# -m 1: stop after the first match
PAYLOAD_START=$(grep -n -a -m 1 '^__PAYLOAD_BELOW__$' "$0" |
cut -d: -f1)

# Use tail to get all content *after* the payload marker line.
# The payload itself is piped to a decoder/unpacker.
# In this example, it's a compressed tarball.
tail -n +${(PAYLOAD_START + 1)} "$0" | tar -xzf -C /tmp/

echo "Payload extracted to /tmp/"
ls -l /tmp/

# It's crucial to exit here so the shell doesn't try to execute
the binary payload.
exit 0

# This is our marker. Everything after this line is the
payload.
__PAYLOAD_BELOW__
```

2. Prepare your payload. For multiple files, a compressed tarball (.tar.gz) is best.

```
# Create a directory with files to archive
mkdir my_payload
echo "hello" > my_payload/file1.txt
echo "world" > my_payload/file2.txt

# Create the compressed tarball
tar -czf payload.tar.gz my_payload
```

3. Append the payload to the script:

```
# First, copy the script logic to a new file
cp make_archive.sh my_installer.sh
chmod +x my_installer.sh

# Now, append the binary payload to the end of the new script
cat payload.tar.gz >> my_installer.sh
```

Now, ./my_installer.sh is a single file that, when run, will extract the my_payload directory into /tmp.

6. Helper Function for Your Workflow

You can add this function to your .bashrc or .bash_profile to make embedding files even easier.

```
# A function to quickly generate a base64 variable assignment for
a script.
embed_file() {
    # Usage: embed_file <file_to_embed> <bash_variable_name>
    if [ -z "$1" ] || [ -z "$2" ]; then
        echo "Usage: embed_file <file_path> <variable_name>" >&2
        return 1
    fi

    local file_path="$1"
    local var_name="$2"

    if [ ! -f "$file_path" ]; then
        echo "Error: File not found at '$file_path'" >&2
        return 1
    fi

    # Generate the bash code
    printf "%s=$(base64 -w 0 "$file_path")"
    " "$var_name"
}

# --- HOW TO USE IT ---
#
# > embed_file waifu_theme.mp3 WAIFU_MUSIC > embedded_music.sh
#
# This will create a file 'embedded_music.sh' containing:
# WAIFU_MUSIC="SUQzAwAAAAA..."
```